

AA-03

Data Model and Migration Plan

Thirty-six new tables, the shipment split, and eight ordered migrations

30 August 2026 · Rev 1.0 — baseline

EXISTING TABLES

35

NEW TABLES

36

ORDERED MIGRATIONS

Eight

STATE MACHINES

Three

APPEND-ONLY LEDGERS

Three

MONEY TYPE

BIGINT paise

This document is the reference for anyone writing a migration, a query or a repository function. It states every new table and its purpose, the one structural change that makes the rest of the programme possible, the three state machines, how money and personal data are stored, and the eight migrations in the order they must be applied. It assumes AA-00 section 0.6 has been read. Where it names a limitation, that limitation is real and is not scheduled to be fixed inside these ten weeks.

3.1 WHAT CHANGES AND WHAT DOES NOT

Thirty-five tables stay. Thirty-six arrive. Four rules govern all of it.

The live database has 35 tables. This programme adds 36, taking the schema to 71 tables on one Neon primary branch plus one migration-rehearsal branch. Of the existing 35, exactly three are touched, and only by adding columns: **products** gains **partner_org_id** and **moderation_status**, and **orders** gains **legacy_status** as a keepsake of the pre-split value. Nothing else in the existing schema is renamed, retyped, dropped or reordered by any migration in this programme.

That constraint is not tidiness. It follows from the rollback asymmetry set out in AA-01 section 1.9. All three portals are static bundles on Cloudflare Pages, so a bad release rolls back in the time it takes to promote the previous deployment — under a minute, no coordination, no data involved. The api on Cloud Run rolls back the same way, but the database does not: once a destructive migration has run, the previous api revision is pointing at a schema that no longer has the column it reads. The rollback path for the code is instant and the rollback path for the schema is a restore, so the only way to keep a one-command rollback for ten weeks is to never make the schema narrower than the oldest api revision we would want to return to.

The practical rule the team works to: a migration may add, and a later release may stop writing, but nothing is removed inside these ten weeks. Dead columns are the price and section 3.13 says so out loud.

RULE	WHAT IT REQUIRES	WHY THIS AND NOT THE ALTERNATIVE	HOW IT IS CHECKED
R1 Additive only	No column drops, no type narrowing, no renames, no NOT NULL added without a two-step backfill. New columns are nullable or carry a default.	The alternative is expand-and-contract with the contract phase inside the programme. It lost because the contract phase needs a release where no old revision is live, and with four developers shipping daily there is no such window.	CI diffs the migration SQL against a deny list of DROP COLUMN, ALTER TYPE and RENAME and fails the build. The migration-rehearsal gate replays every migration on a restored branch.
R2 created_at everywhere	Every new table carries created_at timestamptz NOT NULL DEFAULT now(). No exceptions, including join tables such as shipment_items and staff_role_grants.	Rejected: relying on the surrogate key order. Sequences tell you insertion order but not when, and three of the new tables are read by date range in the settlement and audit paths where the answer has to be a timestamp.	A Prisma schema lint asserts the field on every model. A migration that omits it fails the schema-drift gate.
R3 Money is BIGINT paise	Every monetary column is BIGINT and named with the suffix _paise. No NUMERIC, no float, no rupee-denominated column anywhere.	NUMERIC(12,2) was rejected because it invites a rounding policy per call site; float was never on the table. Integer paise makes every sum exact and every ledger assertion an equality rather than a tolerance.	A CI grep rejects any migration introducing a NUMERIC or DOUBLE column, and any money column whose name does not end in _paise.
R4 Tenant column is NOT NULL and indexed	Every tenant-owned table carries partner_org_id or fleet_org_id as a NOT NULL foreign key with an index, and it is the leading column of that index.	A nullable tenant column was rejected: it makes scoped() unable to distinguish "not mine" from "belongs to nobody", and one forgotten IS NULL branch becomes a cross-tenant read. delivery_jobs.fleet_org_id is the single deliberate exception and is explained in 3.2.	The tenancy test in AA-02 section 2.5 enumerates every tenant table from the Prisma DMMF and asserts the column, the FK and the index exist.

THE ONE EXCEPTION TO R4, STATED ONCE SO IT IS NEVER A SURPRISE

delivery_jobs.fleet_org_id is nullable, because a job exists before any fleet has accepted it — that is the whole point of the offer wave. A NULL there means unassigned, not unowned, and it is the value the admin exception board searches for. scoped() therefore refuses to serve delivery_jobs through the generic tenant path; the fleet portal reads jobs only through a repository function that requires a non-null fleet_org_id match, and the admin portal reads them through the permission-checked path instead. Any new query that touches delivery_jobs without one of those two entry points is a review failure.

3.2 THE FULL TABLE CATALOGUE

Thirty-six tables in five module groups, with the columns that matter.

What follows is the whole set. It is grouped by the owning backend module from AA-02 section 2.2, and within a group the order is the order the tables are created in, which is also the order the foreign keys allow. The KEY COLUMNS field is not the full column list: it names the primary key, the tenant column, the foreign keys, the status column and every money and timestamp column that a query filters on. Every table also carries created_at per R2, and every table with a status column carries updated_at, and neither is repeated below.

Read this section for what a table is *for*. The shapes that need argument rather than description — the shipment split, the derived order status, the three state machines, the ledgers, the personal-data columns — each get their own section afterwards.

BACKOFFICE

7

identity, roles, saved views

PARTNERS

8

orgs, KYC, money out

FLEET

13

orgs, jobs, offers, POD, COD

ORDERS

3

the split and the timeline

PLATFORM

5

audit, idempotency, notify

TOTAL NEW

36

eight migrations, M1 to M8

3.2.1 Backoffice — identity, authorisation and the things staff save

The backoffice group exists because **staff_users** is a separate table from **users**, and that separation is the single most consequential modelling decision in the group. A customer row and a staff row share almost no columns: a customer has a phone and an address book, a staff member has an email, a TOTP secret and a set of role grants. Merging them behind an is_staff flag was rejected because every customer-facing query would then need to remember the flag, and the failure mode of forgetting it is a staff account appearing in a customer list or, worse, a customer authenticating against an admin audience. Two tables means the four token audiences in AA-02 section 2.4 map to two distinct principal types and a wrong-audience token fails at the table lookup, not at a boolean.

Authorisation is three tables rather than a permissions column on the user. **staff_roles** holds the nine system roles, **role_permissions** holds the grant rows, and **staff_role_grants** joins a user to one or more roles with an optional expiry. The effective permission set is the union of grants across a user's roles minus any DENY row, and DENY wins — that resolution is computed once per request and cached on the request context, never stored on the user. Storing an effective set was rejected: it goes stale the moment a role changes and there is no invalidation story that survives four developers editing roles in a shared staging environment.

TABLE	PURPOSE	KEY COLUMNS	NOTES
staff_users	Internal employee identity	id, email_ci UNIQUE, password_hash, full_name, phone_cipher/last4/index, status, totp_enrolled_at, last_login_at, failed_count, locked_until	status: INVITED ACTIVE SUSPENDED OFFBOARDED. Argon2id, m=19456 t=2 p=1.
staff_roles	Named bundles of admin permissions	id, code UNIQUE, label, is_system, description	9 seeded system roles, editable custom roles.
staff_role_grants	Which staff hold which roles	staff_user_id, staff_role_id, granted_by, granted_at	Composite PK. A staff user may hold several.
role_permissions	Permission entries per role, ALLOW or DENY	staff_role_id, permission, effect	effect: ALLOW DENY. DENY wins across all held roles.
staff_totp	TOTP secrets and recovery codes	staff_user_id, secret_cipher, confirmed_at, recovery_hashes[], last_used_step	last_used_step blocks replay inside the same 30 s window.
saved_views	Per-staff saved filters on list screens	id, staff_user_id, surface, name, query_json, is_shared	Query JSON validated against a Zod allowlist.
exports	Async CSV export jobs	id, requested_by, surface, filters_json, status, row_count, expires_at, download_token_hash	Runs on job_queue. Signed URL valid 15 min. Exports are masked; there is no raw export.

saved_views and **exports** are in this group rather than in platform because both are owned by a staff user and both are meaningless outside the admin portal. An export is a row before it is a file: the row records who asked, for what filter, with which permission, and how many rows came back, capped at 5000 rows. The file itself is generated on demand and never stored, because storing it would mean storing masked-but-still-sensitive data somewhere with no lifecycle and no cost budget. AA-08 covers what an export may contain; the schema's only job here is to make every export attributable.

STAFF_TOTP IS A ROW, NOT A BOOLEAN, AND THAT IS THE WHOLE ENFORCEMENT MECHANISM

There is no `totp_enabled` flag on `staff_users`. A staff member either has a `staff_totp` row with an encrypted secret and a set of bcrypt-hashed recovery codes, or they do not, and without one there is no factor for `requireFreshFactor(300)` to find, so every step-up gated route returns 403 and every money and personal-data route is step-up gated. Modelling it as a flag would have created the state "enabled = true with no usable secret", which is the exact state a half-finished enrolment produces and the exact state that gets waived at 2 am during an incident. The migration that creates the first `SUPER_ADMIN` from `BOOTSTRAP_ADMIN_EMAIL` therefore creates a `staff_users` row with no `staff_totp` row, and the account can do nothing except enrol.

3.2.2 Partners — the supplier tenant, its documents and its money

A partner is an organisation, not a user. **partner_orgs** is the tenant root and its id is the value that appears as `partner_org_id` on every other row a supplier can see. **partner_users** are the people, joined to exactly one org — a user belonging to two orgs is not modelled, because the token carries a single tenant claim and multi-org access would need either a claim array or an org switcher, both of which are AA-02 section 2.5 complications with no MVP customer. Someone who genuinely runs two supplier businesses gets two logins and we accept the friction.

KYC and bank details are separate tables rather than columns on the org for two reasons. First, they have their own lifecycle: **partner_kyc** moves through a review state machine that a compliance reviewer drives, and a rejected submission must stay readable next to the resubmission. Second, they are the densest personal-data rows in the schema, so isolating them means the masking extension in AA-08 has a small, enumerable surface rather than a scattering of cipher columns across a wide org table.

partner_rate_cards is versioned rather than edited. A rate card that is in force when a shipment is priced must remain readable years later, because the commission on a settled payout has to be re-derivable from the card that applied at the time. Editing a card in place was rejected outright: it makes every historical payout unverifiable, which is the one thing a supplier will dispute.

The money end of the group is three tables with three different jobs, and conflating them is the classic mistake. **partner_payouts** is an accrual: one line per shipment, created when the shipment reaches a settleable state, carrying the gross, the commission, the TDS and the net. **payout_runs** is a batch: a weekly envelope that locks a set of accrual lines, gets released by a human with the money-release permission, and then records the bank reference. **partner_ledger** is the statement: an append-only sequence of entries whose sum is what we owe. A single "payouts" table with a paid flag was rejected because a flag cannot express a reversal, and reversals are the normal case in COD.

TABLE	PURPOSE	KEY COLUMNS	NOTES
<code>partner_orgs</code>	A verified product company	<code>id</code> , <code>legal_name</code> , <code>display_name</code> , <code>slug</code> UNIQUE, <code>gst_in_cipher/last4/index</code> , <code>pan_*</code> , <code>status</code> , <code>commission_bps</code> , <code>settlement_cycle</code> , <code>onboarded_at</code> , <code>suspended_reason</code>	<code>status</code> : DRAFT KYC_SUBMITTED KYC_VERIFIED ACTIVE SUSPENDED TERMINATED.
<code>partner_users</code>	An employee of a product company	<code>id</code> , <code>partner_org_id</code> , <code>email_ci</code> , <code>password_hash</code> , <code>full_name</code> , <code>phone_*</code> , <code>role_code</code> , <code>status</code> , <code>totp_enrolled_at</code> , <code>invited_by</code>	UNIQUE (<code>partner_org_id</code> , <code>email_ci</code>). Four role codes.
<code>partner_kyc</code>	Submitted KYC documents and the review trail	<code>id</code> , <code>partner_org_id</code> , <code>doc_type</code> , <code>media_public_id</code> , <code>status</code> , <code>reviewed_by</code> , <code>reviewer_note</code> , <code>reviewed_at</code>	<code>doc_type</code> : GST_CERT PAN LICENCE_SEED LICENCE_PESTICIDE LICENCE_FERTILISER BANK_PROOF ADDRESS_PROOF.
<code>partner_bank_accounts</code>	Where settlement is paid	<code>id</code> , <code>partner_org_id</code> , <code>account_cipher/last4/index</code> , <code>ifsc_cipher/last4</code> , <code>holder_name</code> , <code>verified_at</code> , <code>is_primary</code>	Verified by a manual ₹1 admin check at MVP. No penny-drop API.
<code>partner_rate_cards</code>	Commission by category, effective-dated	<code>id</code> , <code>partner_org_id</code> , <code>category_id</code> NULL, <code>commission_bps</code> , <code>effective_from</code> , <code>effective_to</code>	NULL category is the org default. Never edited in place; superseded by a new row.

TABLE	PURPOSE	KEY COLUMNS	NOTES
partner_payouts	One settlement line per shipment	id, partner_org_id, shipment_id, gross_paise, commission_paise, tds_paise, adjustment_paise, net_paise, payout_run_id, status	Append-only. status: ACCRUED IN_RUN PAID WITHHELD.
payout_runs	A weekly batch across many payout lines	id, kind, period_start, period_end, total_paise, line_count, status, released_by, released_at, utr_reference_cipher/last4	kind: PARTNER FLEET. status: DRAFT LOCKED RELEASED SETTLED FAILED.
partner_ledger	Append-only running balance per org	id, partner_org_id, entry_type, amount_paise, balance_after_paise, ref_type, ref_id, created_at	Postgres rule blocks UPDATE and DELETE. Mirrors wallet_ledger discipline.

3.2.3 Fleet — the delivery tenant, and the largest group for a reason

Thirteen of the thirty-six new tables belong to the fleet module. That is not scope creep; it is the cost of the delivery layer being a marketplace rather than a field in the order. A fleet org has the same tenant shape as a partner org — org, users, KYC, bank account, ledger — and then the tables that exist only here: the operational assets (**riders**, **vehicles**), the coverage declaration (**fleet_service_areas**), the work (**delivery_jobs**), the allocation mechanism (**job_offers**), and the evidence (**pod_records**, **cod_remittances**).

fleet_service_areas stores pincodes, not polygons. A geometry column would need PostGIS, which is available on Neon but adds an extension to the migration path and a spatial index to the storage budget for a matching problem that is currently "does this fleet serve 751024". Pincode rows with a serviceable flag answer that with an integer index. The limitation is real and named in AA-07: a fleet that covers half a pincode has to either accept the whole one or decline jobs, and declines are counted against it.

riders and **vehicles** are owned by the fleet org, and a rider is not a login. Riders in this programme do not have portal accounts; the fleet dispatcher acts on their behalf and the rider's phone number lives as cipher plus last four like any other personal datum. The constraint RIDER_MAX_ACTIVE_JOBS = 8 is enforced by a counting query at assignment time, not by a column on the rider: a cached active_jobs integer needs decrementing on six different job transitions and every missed decrement permanently reduces a rider's capacity, with no symptom other than jobs quietly failing to assign.

TABLE	PURPOSE	KEY COLUMNS	NOTES
fleet_orgs	A registered third-party delivery company	id, legal_name, display_name, slug UNIQUE, gstin_*, pan_*, status, tier, acceptance_sla_minutes, onboarded_at, suspended_reason	tier 1..3 drives broadcast order. status mirrors partner_orgs.
fleet_users	An employee or rider of a delivery company	id, fleet_org_id, email_ci NULL, phone_*, password_hash, full_name, role_code, status, totp_enrolled_at	Riders may log in by mobile OTP instead of password.
fleet_kyc	Company KYC documents and review trail	id, fleet_org_id, doc_type, media_public_id, status, reviewed_by, reviewer_note	doc_type adds TRANSPORT_LICENCE and GOODS_INSURANCE to the partner set.
fleet_bank_accounts	Where delivery fees are paid	id, fleet_org_id, account_cipher/last4/index, ifsc_*, holder_name, verified_at, is_primary	Same manual verification path as partners.
fleet_service_areas	Which pincodes a fleet serves, and at what price	id, fleet_org_id, pincode, max_weight_g, cod_allowed, base_fee_paise, per_km_paise, cod_fee_bps, priority, active	UNIQUE (fleet_org_id, pincode). Admin approves changes.
riders	An individual delivery person	id, fleet_org_id, fleet_user_id NULL, full_name, phone_*, dl_cipher/last4/index, dl_expiry, status, active_job_count, rating_avg	status: PENDING_DOCS ACTIVE ON_LEAVE SUSPENDED. Hard cap of 8 concurrent active jobs.
vehicles	A vehicle a rider may use	id, fleet_org_id, reg_cipher/last4/index, kind, capacity_g, insurance_expiry, active	kind: TWO_WHEELER THREE_WHEELER PICKUP TRUCK. Heavy equipment needs PICKUP or TRUCK.
delivery_jobs	One delivery attempt lifecycle per shipment	id, code UNIQUE, shipment_id UNIQUE, fleet_org_id NULL, rider_id NULL, vehicle_id NULL, status, pickup_pincode, drop_pincode, weight_g, cod_amount_paise, fee_quote_paise, promised_by, accepted_at, picked_up_at, delivered_at, attempt_count	Eleven states. fleet_org_id is NULL until an offer is accepted.
job_offers	A broadcast offer to one fleet with a TTL	id, delivery_job_id, fleet_org_id, wave, offered_at, expires_at, status, responded_at, decline_reason	status: OFFERED ACCEPTED DECLINED EXPIRED WITHDRAWN. UNIQUE (job, org, wave).

TABLE	PURPOSE	KEY COLUMNS	NOTES
pod_records	Proof of delivery for one attempt	id, delivery_job_id, attempt_no, outcome, otp_hash, geo_lat, geo_lng, geo_accuracy_m, distance_m, photo_public_id NULL, receiver_name, captured_at, captured_by_rider_id	outcome: DELIVERED CUSTOMER_UNAVAILABLE REFUSED ADDRESS_INVALID DAMAGED. OTP hash only.
cod_remittances	Cash a fleet owes the platform	id, fleet_org_id, period_start, period_end, expected_paise, declared_paise, received_paise, variance_paise, status, reference_cipher/last4, reconciled_by, reconciled_at	status: OPEN DECLARED IN_TRANSIT RECEIVED RECONCILED SHORT DISPUTED.
fleet_payouts	One earnings line per completed job	id, fleet_org_id, delivery_job_id, base_paise, distance_paise, cod_fee_paise, penalty_paise, net_paise, payout_run_id, status	Append-only. Mirrors partner_payouts.
fleet_ledger	Append-only balance per fleet org, net of COD held	id, fleet_org_id, entry_type, amount_paise, balance_after_paise, ref_type, ref_id	A fleet that is short on COD carries a negative balance and is blocked from new offers.

pod_records stores one row per attempt, not one per job, which is why attempt_no is part of its identity. A failed attempt is evidence too: it carries the outcome code, the rider's coordinates and the distance from the delivery pin, and the POD_GEOFENCE_METRES = 200 check is evaluated against distance_m at capture time. The delivery OTP is stored as otp_hash and nothing else — a bcrypt hash, never the digits, never in a log line, and there is no query anywhere that reads it except the verify path. **cod_remittances** gets its own section at 3.7 because its arithmetic decides whether a fleet keeps receiving work.

ATV-SELF IS A ROW IN FLEET_ORGS, NOT A SPECIAL CASE IN CODE

M5 seeds one internal fleet org, ATV-SELF, at tier 1. It exists so that on the day the fleet module ships and no third-party fleet has completed KYC, orders can still be delivered: staff assign the job to ATV-SELF and the same delivery_jobs, pod_records and cod_remittances rows are written as for anyone else. The alternative — a nullable fleet_org_id meaning "delivered by us" — was rejected because it forks every fleet query into a self branch and an external branch, and the self branch is the one nobody tests. The cost of the chosen shape is that ATV-SELF appears in fleet league tables and in the settlement run, so both have to exclude it by org id rather than pretend it is not there.

3.2.4 Orders — three tables that change the shape of the system

Only three tables, and they are the reason the other thirty-three can exist. **shipments** introduces the level between an order and a delivery; **shipment_items** says which order lines belong to which shipment; **order_events** becomes the single timeline that the customer app reads. Section 3.3 walks the split entity by entity and section 3.4 explains why orders.status stops being a column anyone writes.

One point that belongs here rather than there: **order_events** carries a nullable shipment_id and it is the only event table in the schema. Job transitions, shipment transitions and order-level events all land in it, distinguished by event_code and by whether shipment_id is set. A separate job_events table was considered and rejected: the customer tracker and the admin order timeline both want one chronological list, and building that from two tables means a UNION with a synthetic sort key on every read of the busiest screen in the app. The cost of the decision is that order_events is the fastest-growing table in the database and it is the one named first in section 3.13.

TABLE	PURPOSE	KEY COLUMNS	NOTES
shipments	The part of an order fulfilled by one supplier from one place	id, code UNIQUE, order_id, partner_org_id, status, item_count, weight_g, goods_paise, cod_paise, ready_by, packed_at, handed_over_at, delivered_at	THE structural change. An order has 1..n shipments; each has exactly one delivery_job.
shipment_items	Order lines assigned to a shipment	shipment_id, order_item_id, qty	Composite PK. Sum of qty per order_item must equal ordered qty.
order_events	Append-only timeline behind the customer tracker	id, order_id, shipment_id NULL, event_code, actor_kind, actor_id, payload_json, created_at	The single source for the app tracker; the app never reads shipments directly.

3.2.5 Platform — the tables that make the rest auditable

These five are cross-cutting: no module owns them and every module writes to them. **pii_disclosures** is the register behind the single manifest exception and gets section 3.9. **audit_log** records every mutating admin request with a before/after diff, capped at `AUDIT_DIFF_MAX_BYTES = 8192` bytes so that one bulk update cannot write a megabyte into a 0.5 GB database. The diff passes through the masking extension before storage, which means the audit log itself never contains plaintext personal data — a property worth stating because an audit trail is the most common place a masking regime leaks.

idempotency_keys exists because Razorpay webhooks retry, fleets double-tap accept on a bad connection, and a settlement release must never pay twice. The unique constraint is on (actor_kind, actor_id, endpoint, key) and the stored request_hash makes a replay with the same key but a different body a 409 rather than a silent re-run of the wrong thing. Rows expire after 24 hours and a cron deletes them; this is the one new table with a delete path, and it is safe because a key older than a day cannot be part of a live retry.

notification_templates and **webhook_deliveries** are the smallest and least interesting rows in the schema and the first ones anyone will want to skip. They earn their place because WhatsApp template text has to be approved by Meta before it is sent, which means the exact string must be stored and versioned rather than assembled in code, and because a failed webhook delivery that is not recorded is a payment we never learned about.

TABLE	PURPOSE	KEY COLUMNS	NOTES
pii_disclosures	Register of every unmasked read of customer data	id, actor_kind, actor_id, subject_user_id, field, reason_code, ref_type, ref_id, created_at	Written inside the same transaction as the read. Retained 24 months. Never purged by a job.
audit_log	Every mutating admin request with a before/after diff	id, staff_user_id, permission, method, path, target_type, target_id, diff_json, ip_hash, user_agent, created_at	Diff values pass through the masking extension before storage.
idempotency_keys	Replay protection for money and offer endpoints	id, actor_kind, actor_id, endpoint, key, request_hash, response_status, response_body, created_at, expires_at	UNIQUE (actor_kind, actor_id, endpoint, key). 24 h TTL.
notification_templates	WhatsApp and FCM template registry	id, code UNIQUE, channel, language, body, variables[], approved_at, meta_template_name	WhatsApp bodies must match the Meta-approved template exactly or the send fails.
webhook_deliveries	Outbound webhooks to partner and fleet systems	id, subscriber_kind, subscriber_org_id, event_code, payload_json, attempt, status, next_attempt_at, signature_key_id	Phase 2 feature; table created in migration 8, unused at MVP.

WHERE THE COUNTS COME FROM

35 existing plus 36 new is 71 tables, which is the figure AA-09 uses in the Neon budget line. The split is 7 backoffice, 8 partners, 13 fleet, 3 orders and 5 platform. If a table appears in a migration in section 3.11 that is not in this catalogue, the catalogue is wrong and the fact base is the thing to fix.

TWO INCONSISTENCIES IN THE SOURCE OF TRUTH, NAMED NOT HIDDEN

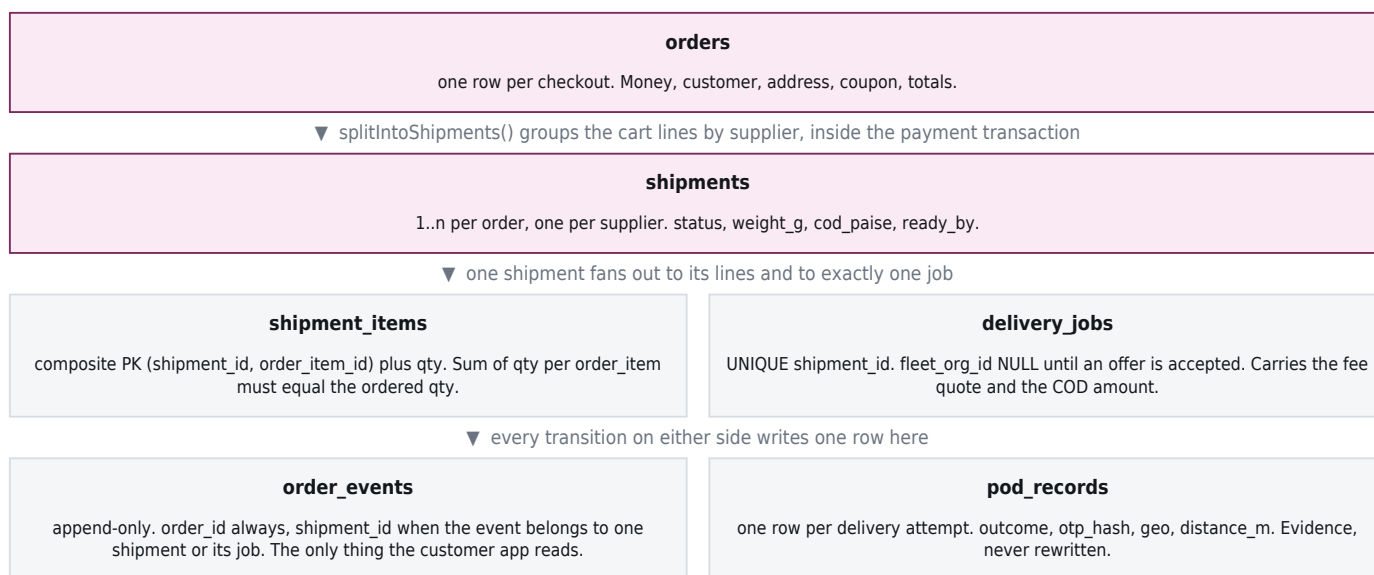
The catalogue entry for `delivery_jobs` says eleven states while the job state machine in section 3.5 enumerates 13, and the status comment on `cod_remittances` lists seven values while the remittance machine that operations actually drives has five. Both are wording drift in the fact base, not two implementations. The state-machine tables in 3.5 are authoritative and the enum comments must be corrected before M6 and M7 are written, because the FSM table test in CI reads the machine, not the comment.

3.3 THE SHIPMENT SPLIT

The one structural change. An order has 1..n shipments; each shipment has exactly one job.

Before this programme an order was a single fulfilment unit. It had one status, one set of pack and delivery timestamps, and an implicit assumption that one person picked it, packed it and delivered it. That assumption is false the moment a customer puts seed from one supplier and a sprayer from another into the same cart, which is the normal case in a marketplace and was already happening before the supplier portal existed — it was just being resolved by a human on WhatsApp.

The split introduces one level. An order is still the customer's unit: one payment, one invoice, one delivery address, one coupon. A **shipment** is the supplier's unit: the part of that order which one supplier fulfils from one place. A **delivery_job** is the fleet's unit, and there is exactly one per shipment — enforced by a UNIQUE constraint on `delivery_jobs.shipment_id`, not by convention.



Read the diagram downwards as ownership and upwards as aggregation. Nothing in the lower rows is visible to a customer directly; the app reads `order_events` and the order row, which is why the split did not require a mobile release.

3.3.1 Entity by entity: what moved, what stayed, what became derived

CONCERN	BEFORE THE SPLIT	AFTER THE SPLIT	WHY IT LANDED THERE
Supplier identity	Not expressible. An order had no supplier column; the supplier was inferred by joining through <code>order_items</code> to products.	<code>shipments.partner_org_id</code> , NOT NULL, indexed. One shipment belongs to exactly one supplier.	It is the tenant column the supplier portal scopes on. Inferring it through a join means every supplier query is a three-table join whose WHERE clause is the security boundary.
Fulfilment status	<code>orders.status</code> , written directly by the pack and dispatch actions.	<code>shipments.status</code> , driven by the 16-state machine in 3.5. <code>orders.status</code> is derived.	Two suppliers can be in different states on the same order. One column cannot hold two answers, and picking the worse of the two is exactly what <code>reduceOrderStatus()</code> now does deliberately instead of accidentally.
Pack and handover timestamps	<code>orders.packed_at</code> , <code>orders.handed_over_at</code> .	<code>shipments.packed_at</code> , <code>shipments.handed_over_at</code> . The order-level columns keep their last written value and are no longer read.	These are supplier actions. Additive-only means they are not dropped from orders; they become dead columns, which is the trade named in 3.13.
Delivery outcome	<code>orders.delivered_at</code> , set once by whoever closed the order.	<code>shipments.delivered_at</code> plus <code>delivery_jobs.delivered_at</code> and <code>attempt_count</code> , with <code>pod_records</code> holding the evidence per attempt.	Delivery is a fleet action with retries. A single order timestamp cannot represent two failed attempts followed by a success on one of two shipments.
Weight	Not stored. Shipping was a flat fee.	<code>shipments.weight_g</code> , set at pack time and used to quote the fleet fee.	Weight is a property of a physical parcel, and a parcel is a shipment. An order-level weight is only ever a sum and is derived.

CONCERN	BEFORE THE SPLIT	AFTER THE SPLIT	WHY IT LANDED THERE
COD amount	orders.cod_paise for the whole order.	shipments.cod_paise, apportioned across shipments; the order total is unchanged.	Cash is collected per parcel by the rider who carries it, and the fleet owes us per parcel. Apportionment is by goods value, with the rounding remainder placed on the largest shipment so the sum is exact.
Line assignment	order_items belonged to the order and nothing else.	shipment_items (shipment_id, order_item_id, qty).	A qty column rather than a straight foreign key on order_items, because one line of five bags can in principle be split. That flexibility has a cost, stated in 3.13.
Payment, coupon, pricing snapshot, customer, address, order code	On orders.	Still on orders. Untouched by any migration in this programme.	These are properties of the purchase, not of a parcel. One payment and one invoice per order is the customer promise the split was designed not to break.

3.3.2 The three columns on orders that became derived

Three columns survive on orders but are no longer written by any handler. They are recomputed by `recomputeOrder(orderId)` inside the same transaction as the shipment transition that changed them, and nothing else may assign them. This is a projection kept in the row it describes, not a cache with its own lifecycle.

orders.status	Output of <code>reduceOrderStatus()</code> over the order's shipment statuses. Section 3.4 is the rule table. Writing it directly is invariant I-11 and is caught by an ESLint rule plus a repository guard that throws <code>DERIVED_FIELD_WRITE</code> .
orders.delivered_at	<code>MAX(shipments.delivered_at)</code> once every shipment is in a terminal delivered or returned state, <code>NULL</code> before that. A partially delivered order has no delivery date, which is the honest answer and the one the invoice needs.
orders.weight_g	<code>SUM(shipments.weight_g)</code> over shipments that have been packed, <code>NULL</code> while none have. It exists only so the admin order list can sort by weight without a join.

```
-- The two assertions that must hold for every order after the split. Both return zero rows.
-- 1. COD apportionment is exact.
SELECT o.id, o.cod_paise, SUM(s.cod_paise) AS shipment_sum
FROM orders o JOIN shipments s ON s.order_id = o.id
GROUP BY o.id, o.cod_paise
HAVING SUM(s.cod_paise) <> o.cod_paise;

-- 2. Every ordered line is fully assigned to shipments, no more and no less.
SELECT oi.id, oi.qty, COALESCE(SUM(si.qty), 0) AS assigned
FROM order_items oi LEFT JOIN shipment_items si ON si.order_item_id = oi.id
GROUP BY oi.id, oi.qty
HAVING COALESCE(SUM(si.qty), 0) <> oi.qty;
```

THE ALTERNATIVE THAT LOST: ONE DELIVERY PER ORDER, AND SPLIT THE CART AT CHECKOUT

The cheaper design was to keep the existing one-delivery-per-order model and make a multi-supplier cart create several orders at checkout, one per supplier. It was rejected for three reasons, in order of weight. First, it breaks the customer promise: one basket becomes three payments or one payment mapped to three orders, three invoices, three tracking screens, and a refund of "my order" becomes three refunds. Second, the mobile app renders one order per checkout and one tracker per order; changing that is an app release through store review, and a mobile release is explicitly out of scope for these ten weeks. Third, coupon and shipping-fee apportionment across sibling orders has no non-arbitrary answer, and whichever answer we picked would be visible on an invoice a supplier or a customer could dispute. The shipment level costs three new tables and one risky migration; the alternative cost a store release and a permanently confusing invoice.

3.4 ORDER STATUS AS A PROJECTION

Invariant I-11: `orders.status` is computed, in this order, and never assigned.

`reduceOrderStatus(shipmentStatuses, paymentState)` is a pure function. It takes the statuses of an order's shipments and returns one of the 10 values in `ORDER_STATUS`. The rules below are evaluated top to bottom and the first match wins, so the order of the table is part of the specification, not a presentation choice. AA-02 section 2.8 covers where the function is called from; this section is the rule set and the arithmetic.

The reason for a projection rather than a column anyone can write is that the interesting states are disagreements between shipments, and a writable column resolves a disagreement wherever the last writer happened to be. With ten rules in a fixed order there is exactly one answer, it is testable without a database, and the CI table test enumerates every reachable combination.

#	CONDITION (FIRST MATCH WINS)	<code>orders.status</code> BECOMES	IN THE WORKED EXAMPLE BELOW
1	All shipments CANCELLED	CANCELLED	Does not match. Neither shipment is CANCELLED.
2	All shipments DELIVERED	DELIVERED	Does not match. Shipment B is not DELIVERED.
3	All shipments terminal, mix of DELIVERED and RETURNED/RTO_DELIVERED	PARTIALLY_DELIVERED	Does not match. Shipment B is not terminal, so this is not an all-terminal mix.
4	All shipments RETURNED or RTO_DELIVERED	RETURNED	Does not match. Neither shipment is RETURNED or RTO_DELIVERED.
5	Any shipment in RETURN_REQUESTED..RETURN_PICKED	RETURN_IN_PROGRESS	Does not match. Shipment B is not in a return state.
6	Any shipment OUT_FOR_DELIVERY or FAILED_ATTEMPT	OUT_FOR_DELIVERY	Fires. Shipment B is OUT_FOR_DELIVERY, so the order is OUT_FOR_DELIVERY.
7	Any shipment HANDED_OVER, IN_TRANSIT or RTO_INITIATED	DISPATCHED	Not evaluated. Rule 6 already matched.
8	Any shipment PACKING or READY_FOR_PICKUP	PACKED	Not evaluated. Rule 6 already matched.
9	Any shipment AWAITING_ACCEPTANCE or ACCEPTED	CONFIRMED	Not evaluated. Rule 6 already matched.
10	Payment not captured and not COD-confirmed	PENDING_PAYMENT	Not evaluated. Rule 6 already matched.

3.4.1 Worked example: one order, two shipments, two different truths

Order ATV-2611-0417 contains four lines from two suppliers. `splitIntoShipments()` created SHP-ATV-2611-0417-A against Sagar Agro Traders and SHP-ATV-2611-0417-B against Krishi Bhandar at payment capture. Shipment A was delivered at 14:22 with a POD row and an OTP match; shipment B moved to OUT_FOR_DELIVERY at 15:05 and is on a rider's bike.

Evaluation walks rules 1 to 6 as shown in the fourth column above and stops. Rule 3 is the one worth pausing on: PARTIALLY_DELIVERED exists precisely for a mixed *terminal* outcome, so while shipment B is still moving the customer is never shown "partially delivered". Rule 6 matches and `orders.status` becomes OUT_FOR_DELIVERY.

WHAT THE READER SEES	VALUE	WHY
<code>orders.status</code>	OUT_FOR_DELIVERY	Rule 6. Recomputed by <code>recomputeOrder()</code> in the same transaction as B's transition.
<code>orders.delivered_at</code>	NULL	Not every shipment is terminal. A partly delivered order has no delivery date.
App tracker, order level	Out for delivery	Reads <code>order_events</code> plus the order row. One chip, taken from <code>orders.status</code> .
App tracker, parcel level	Two rows: A delivered 14:22, B out for delivery	<code>order_events</code> rows carry <code>shipment_id</code> , so the tracker groups by parcel without reading shipments.
Supplier portal, Sagar Agro	Shipment A: DELIVERED, payout accrued	<code>scoped()</code> restricts to <code>partner_org_id</code> . Sagar Agro cannot see that shipment B exists.
Supplier portal, Krishi Bhandar	Shipment B: OUT_FOR_DELIVERY	Same mechanism, opposite row. Neither supplier ever learns the order total.
Admin order detail	Order OUT_FOR_DELIVERY with both shipments and both jobs listed	The only view that sees the whole order, and it is permission-checked, not tenant-scoped.

3.5 THE THREE STATE MACHINES

Shipment (16 states), delivery job (13) and COD remittance (five).

All three machines are declared as data in packages/shared and enforced by one function, `assertTransition(machine, from, to, actorKind)`, which is invariant I-05. Nothing writes a status column without passing through it. An attempt to move to a state that is not in the from-state's next list, or to move a state a caller of that actor kind may not drive, throws and the handler returns 409 `INVALID_TRANSITION` with both states in the error body. The transition and the `order_events` row are written in one transaction, so an event without a state change, or a state change without an event, does not exist.

Declaring the machines as tables rather than as switch statements is what makes the CI FSM table test possible: it walks every state, attempts every one of the other states as a target, and asserts that exactly the declared set succeeds. That test is the reason the tables below can be trusted as a specification rather than documentation.

3.5.1 shipments.status — the supplier and fleet lifecycle

STATE	MAY MOVE TO	WHO MAY DRIVE IT	NOTES
AWAITING_ACCEPTANCE	ACCEPTED, CANCELLED	system, staff	Created when payment is captured or COD is confirmed. Supplier has not yet acknowledged.
ACCEPTED	PACKING, CANCELLED	partner, staff	Supplier acknowledged and committed to <code>ready_by</code> . Stock is already reserved.
PACKING	READY_FOR_PICKUP, CANCELLED	partner	Physically being assembled. The only state a supplier may still short-ship from.
READY_FOR_PICKUP	HANDED_OVER, CANCELLED	partner	Triggers job offer broadcast. A shipment sitting here past <code>ready_by</code> raises an ops alert.
HANDED_OVER	IN_TRANSIT	fleet	Rider scanned the shipment code at the supplier gate. Custody has moved.
IN_TRANSIT	OUT_FOR_DELIVERY, RTO_INITIATED	fleet	Between hubs or on the road.
OUT_FOR_DELIVERY	DELIVERED, FAILED_ATTEMPT	fleet	Address unmasked for the rider from this point back to <code>ACCEPTED</code> , and logged.
FAILED_ATTEMPT	OUT_FOR_DELIVERY, RTO_INITIATED	fleet, staff	Three failed attempts force <code>RTO_INITIATED</code> automatically.
DELIVERED	RETURN_REQUESTED	system	Terminal for fulfilment. Accrues <code>partner_payouts</code> and <code>fleet_payouts</code> .
RETURN_REQUESTED	RETURN_PICKED, RETURN_REJECTED	customer, staff	Customer raised a return with photo evidence inside the return window.
RETURN_PICKED	RETURNED	fleet	Reverse job completed; goods with the supplier.
RETURNED	— terminal	staff	Reverses the payout accrual with a compensating ledger entry, never an <code>UPDATE</code> .
RTO_INITIATED	RTO_DELIVERED	fleet, staff	Undeliverable; heading back to the supplier.
RTO_DELIVERED	— terminal	fleet	Supplier confirmed receipt. Customer refunded in full.
RETURN_REJECTED	— terminal	staff	Evidence insufficient. Customer notified with reason.
CANCELLED	— terminal	customer, partner, staff	Releases reserved stock with the inverse conditional statement.

3.5.2 delivery_jobs.status — the allocation and custody lifecycle

STATE	MAY MOVE TO	WHO MAY DRIVE IT	NOTES
DRAFT	OFFERED	system	Created with the shipment. No fleet attached.
OFFERED	ACCEPTED, UNASSIGNED	system	One or more <code>job_offers</code> live. First accept wins; the rest are <code>WITHDRAWN</code> in the same transaction.
UNASSIGNED	OFFERED, ASSIGNED_MANUAL	system, staff	All waves exhausted. Sits on the admin exception board.

STATE	MAY MOVE TO	WHO MAY DRIVE IT	NOTES
ASSIGNED_MANUAL	ACCEPTED	staff	Ops assigned a fleet directly, bypassing the auction.
ACCEPTED	PICKED_UP, CANCELLED	fleet	Fleet owns the job. A rider must be attached before PICKED_UP.
PICKED_UP	IN_TRANSIT, CANCELLED	fleet	Scan at the supplier gate.
IN_TRANSIT	OUT_FOR_DELIVERY, RTO_INITIATED	fleet	Moving.
OUT_FOR_DELIVERY	DELIVERED, FAILED_ATTEMPT	fleet	Manifest unmask window is open.
FAILED_ATTEMPT	OUT_FOR_DELIVERY, RTO_INITIATED	fleet	Requires a pod_records row with an outcome.
DELIVERED	— terminal	fleet	Requires OTP match and a POD inside the 200 m geofence.
RTO_INITIATED	RTO_DELIVERED, CANCELLED	fleet, staff	Reverse leg.
RTO_DELIVERED	— terminal	fleet	Confirmed back with the supplier.
CANCELLED	— terminal	staff	Only staff may cancel a job already accepted.

The two machines run in parallel on the same parcel and are deliberately not merged. A shipment can be `READY_FOR_PICKUP` while its job is still `OFFERED`, `UNASSIGNED` or `ASSIGNED_MANUAL` — the supplier has finished and nobody has agreed to carry it, which is precisely the situation the admin exception board exists to surface. Merging them into one status would have made that state unnameable. The states that must stay in step are the custody ones: a job reaching `PICKED_UP` writes `shipments.status = HANDED_OVER` in the same transaction, and a job reaching `DELIVERED` writes `shipments.status = DELIVERED`. Those two pairs are the only coupling, and each is a single function.

3.5.3 `cod_remittances.status` — the cash machine, five states

The third machine governs money that exists as physical cash in a rider's bag and therefore cannot be reconciled by a webhook. It has five states and, unlike the other two, most of its transitions are driven by a human looking at a bank statement.

STATE	MAY MOVE TO	WHO MAY DRIVE IT	NOTES
DUE	DECLARED	system	Opened by the weekly cron for a fleet with delivered COD shipments in the period. <code>expected_paise</code> is the sum over <code>pod_records-confirmed</code> COD deliveries and is not editable by anyone.
DECLARED	DEPOSITED, SHORT	fleet	The fleet states what it is banking and against which reference. <code>declared_paise</code> may differ from <code>expected_paise</code> ; that difference alone does not block anything yet.
DEPOSITED	RECONCILED, SHORT	staff	Finance has seen the credit and entered <code>received_paise</code> and the bank reference, which is stored as cipher plus last four like any other reference.
RECONCILED	— terminal	staff	<code>received_paise</code> matches <code>expected_paise</code> , or the variance is inside <code>COD_VARIANCE_BLOCK_BPS = 200 bps</code> and has been accepted. Writes the settling <code>fleet_ledger</code> entry. Nobody may leave this state.
SHORT	— terminal	staff	Variance exceeds the threshold, or the deposit never arrived. Blocks the fleet from new offers and leaves a negative <code>fleet_ledger</code> balance. Resolution is a new remittance row plus a compensating ledger entry, never an edit of this one.

3.5.4 Illegal transitions, and how each is actually rejected

MACHINE	A TRANSITION SOMEONE WILL TRY	WHAT HAPPENS AND WHY
shipment	<code>AWAITING_ACCEPTANCE</code> to <code>READY_FOR_PICKUP</code> , skipping <code>ACCEPTED</code> and <code>PACKING</code>	409 <code>INVALID_TRANSITION</code> . <code>READY_FOR_PICKUP</code> is not in the next list. It matters because <code>ACCEPTED</code> is where <code>ready_by</code> is committed and <code>PACKING</code> is the only state a short-ship is legal from; skipping both loses the promise and the audit of what was actually packed.

MACHINE	A TRANSITION SOMEONE WILL TRY	WHAT HAPPENS AND WHY
shipment	DELIVERED to CANCELLED, after the customer refuses at the door	409 INVALID_TRANSITION. Custody has already moved and money has accrued. The legal path is DELIVERED to RETURN_REQUESTED, which reverses the accrual with a compensating ledger row.
shipment	A partner driving HANDED_OVER, marking a parcel dispatched itself	403. The actor kind is checked as well as the state pair: HANDED_OVER is fleet-driven because it is the moment custody transfers, and a supplier asserting it would let a parcel that never left the shop count as dispatched against the ready_by alert.
job	OFFERED to PICKED_UP, a fleet scanning before accepting	409 INVALID_TRANSITION. ACCEPTED is the state that closes the auction and withdraws the sibling offers; skipping it leaves live offers on a job already in progress and two fleets able to accept.
job	OUT_FOR_DELIVERY to DELIVERED with no POD row	The state pair is legal, so the transition is refused one layer up: the deliver handler requires an OTP match and a pod_records row inside the 200 m geofence, and returns 422 POD_REQUIRED otherwise. This is the one case where the machine alone is not the guard, and it is called out here so nobody assumes assertTransition() is sufficient.
job	A fleet cancelling a job it has already accepted	403. CANCELLED is staff-only from ACCEPTED onwards. A fleet that wants out raises a release request, which staff action; otherwise a fleet could shed every job it disliked after winning it and the acceptance rate metric would mean nothing.
COD remittance	DUE straight to RECONCILED, closing a week nobody banked	409 INVALID_TRANSITION. DECLARED and DEPOSITED are the two rows of evidence. Without them a reconciliation is an assertion that cash arrived with nothing recording who said so.
COD remittance	SHORT back to DECLARED, to fix a mistyped amount	409. Both terminal states are absorbing. A mistyped received_paise is corrected by a new remittance row and a compensating fleet_ledger entry, which is the same discipline as section 3.6 and for the same reason: the wrong number stays visible.

3.6 MONEY TABLES: THREE APPEND-ONLY LEDGERS

Every amount is BIGINT paise. Nothing is ever updated. Balance is a sum.

There are three ledgers and they have the same shape on purpose: an id, a tenant or user column, an entry_type, a signed amount_paise, a materialised balance_after_paise, a (ref_type, ref_id) pointer to whatever caused the entry, a nullable reverses_ledger_id, and created_at. A developer who has read one has read all three. wallet_ledger already exists and sets the discipline; M7 adds the other two and makes all three immutable at the database rather than in the service layer.

Signed amounts with one entry per event were chosen over paired debit and credit columns. Double-entry with two rows per event is the accounting-correct model and it lost on cost: it doubles the row count in the table most likely to exhaust the 0.5 GB Neon budget, and the thing double-entry buys — a trial balance across accounts — is not a report anyone in this programme asks for. The tradeoff is real: these ledgers can tell you what we owe one supplier and cannot tell you that the platform's books balance.

LEDGER	ONE SEQUENCE PER	ENTRY TYPES	WHAT THE SUM MEANS
wallet_ledger	customer (user_id)	REFUND_CREDIT, PROMO_CREDIT, ORDER_DEBIT, ADJUSTMENT	Spendable store credit. There is no cash-out path and there will not be one: paying it out to a bank account makes the wallet a prepaid payment instrument and needs an RBI PPI licence. Stated in AA-00 and repeated here because it is the first feature anyone asks for.
partner_ledger	supplier org (partner_org_id)	SHIPMENT_ACCRUAL, COMMISSION_DEBIT, TDS_DEBIT, PAYOUT_DEBIT, RETURN_REVERSAL, ADJUSTMENT	What the platform currently owes that supplier. Positive is normal. It goes negative only after a return reversal on an already-paid shipment, which is a real state and is shown as such rather than clamped to zero.
fleet_ledger	fleet org (fleet_org_id)	JOB_ACCRUAL, PENALTY_DEBIT, COD_HELD_DEBIT, COD_SETTLED_CREDIT, PAYOUT_DEBIT, ADJUSTMENT	Delivery earnings minus penalties minus COD cash still held by the fleet. A negative sum means the fleet owes the platform money and the offer engine stops sending it work, which is the mechanism described in 3.7.

```
-- M7. Immutability at the database, because a service-layer rule only binds code we wrote.
CREATE OR REPLACE FUNCTION ledger_immutable() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'LEDGER_IMMUTABLE: % on % is not permitted', TG_OP, TG_TABLE_NAME
    USING ERRCODE = 'P0001';
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER wallet_ledger_immutable BEFORE UPDATE OR DELETE ON wallet_ledger
FOR EACH ROW EXECUTE FUNCTION ledger_immutable();
CREATE TRIGGER partner_ledger_immutable BEFORE UPDATE OR DELETE ON partner_ledger
FOR EACH ROW EXECUTE FUNCTION ledger_immutable();
CREATE TRIGGER fleet_ledger_immutable BEFORE UPDATE OR DELETE ON fleet_ledger
FOR EACH ROW EXECUTE FUNCTION ledger_immutable();

-- The rule form. Kept in the migration file, commented out, and NOT installed:
-- CREATE RULE partner_ledger_no_update AS ON UPDATE TO partner_ledger DO INSTEAD NOTHING;
-- CREATE RULE partner_ledger_no_delete AS ON DELETE TO partner_ledger DO INSTEAD NOTHING;
-- A DO INSTEAD NOTHING rule rewrites the statement away, so the BEFORE trigger never fires
-- and the write fails silently with UPDATE 0. The two are mutually exclusive, not layered.
```

3.6.1 Balance: a sum, with a materialised answer next to it

The authoritative balance of any ledger is the sum of its amount_paise. There is no balances table and no mutable balance field on an org, because a stored balance is a second source of truth that drifts the first time a transaction commits half way. What each row does carry is balance_after_paise: the running total as at that row, written inside the same transaction that inserts it, under a row-level lock on the org so two concurrent entries cannot compute the same predecessor.

That gives two reads with different costs and the same answer. The statement screen and the settlement run take the fast path — one indexed row. Any audit, and a nightly cron, take the slow path and assert the two agree. This resolves the apparent contradiction between "balances are never stored" in AA-00 and a balance column in the catalogue: the column is a derivable snapshot that a query can prove wrong, not a value anyone trusts on its own.

```
-- Fast path: what the supplier statement header and the settlement run use.
SELECT balance_after_paise FROM partner_ledger
  WHERE partner_org_id = $1 ORDER BY id DESC LIMIT 1;          -- one row, index-only

-- Slow path: the authoritative value, and the nightly assertion that they agree.
SELECT COALESCE(SUM(amount_paise), 0) FROM partner_ledger WHERE partner_org_id = $1;

-- The cron that must return zero rows. If it ever does not, the fast path is disabled by
-- config and every balance read falls back to the sum until the cause is found.
SELECT l.partner_org_id, l.id, l.balance_after_paise, s.running
  FROM partner_ledger l
  JOIN (SELECT id, partner_org_id,
              SUM(amount_paise) OVER (PARTITION BY partner_org_id ORDER BY id) AS running
        FROM partner_ledger) s ON s.id = l.id
 WHERE s.running <> l.balance_after_paise;
```

3.6.2 Corrections: a compensating row, never an UPDATE

A wrong entry is not fixed; it is answered. The correction is a new row with the opposite sign, an entry_type ending in _REVERSAL, and reverses_ledger_id pointing at the original. Both rows stay in the statement and the reversal is rendered with a REVERSAL label next to the original's date, so a supplier reading their statement sees that we charged them ₹1,240 in commission on 14 November and gave it back on 16 November, rather than seeing a row that quietly changed value between two visits.

The rule that makes this safe to reason about: reverses_ledger_id may be set at most once per target, enforced by a partial unique index, so an entry cannot be reversed twice. A genuine second correction reverses the reversal, which reads oddly and is correct.

ID	ENTRY TYPE	AMOUNT	balance_after_paise	reverses_ledger_id
4471	SHIPMENT_ACCRUAL	+248000 (₹2,480)	248000	NULL
4472	COMMISSION_DEBIT	-124000 (₹1,240)	124000	NULL
4488	RETURN_REVERSAL	-248000 (₹2,480)	-124000	4471
4489	COMMISSION_REVERSAL	+124000 (₹1,240)	0	4472

WHAT THIS COSTS, SAID PLAINLY

The ledger grows monotonically and a heavily corrected supplier has a long statement. There is no archival path for these three tables in this programme — audit_log has one and the ledgers deliberately do not, because an archived ledger row breaks the running balance chain it is part of. At MVP volumes this is comfortable inside the Neon budget in AA-09; past a few hundred thousand entries per ledger it needs a decision, and the decision will be monthly opening-balance rows rather than deletion.

3.7 COD RECONCILIATION DATA

One row per fleet per week, four money columns, and a 200 bps threshold that stops work.

Cash on delivery is the only money in the system that moves without a webhook. A rider takes notes from a customer, the fleet banks them some days later, and the platform learns about it from a bank statement. `cod_remittances` is the table that makes that gap measurable: one row per fleet org per period, four money columns, and a status from the machine in 3.5.3.

COLUMN	TYPE	WHERE THE NUMBER COMES FROM
<code>expected_paise</code>	BIGINT	Computed by the platform: the sum of <code>shipments.cod_paise</code> over shipments whose job reached DELIVERED with a <code>pod_records</code> row inside the period. No human can edit it. This is the only one of the four the fleet does not supply.
<code>declared_paise</code>	BIGINT	What the fleet says it is banking, entered in the fleet portal when the row moves to DECLARED. A mismatch against <code>expected_paise</code> at this point is information, not yet a fault.
<code>received_paise</code>	BIGINT	What finance can see on the bank statement, entered with the reference when the row moves to DEPOSITED. Nullable until then.
<code>variance_paise</code>	BIGINT	<code>received_paise</code> minus <code>expected_paise</code> . Signed: negative means the platform is short. Stored rather than computed on read so the value that triggered a block is the value on the row.
<code>reference_cipher</code> / <code>reference_last4</code>	TEXT	The bank UTR or deposit slip number. A reference is a personal-data-adjacent identifier and follows the same pattern as everything in 3.8: cipher for storage, last four for display.
<code>reconciled_by</code> / <code>reconciled_at</code>	UUID / timestamptz	The staff user who accepted or rejected the reconciliation. Never a system actor, because closing a cash week is a human decision with a name attached.

```

-- All integer arithmetic. No floats anywhere in the money path.
variance_paise = received_paise - expected_paise -- negative means short
variance_bps   = ROUND(ABS(variance_paise) * 10000 / expected_paise)

block_offers   = (variance_bps > 200) -- COD_VARIANCE_BLOCK_BPS
                OR (SELECT COALESCE(SUM(amount_paise), 0)
                    FROM fleet_ledger WHERE fleet_org_id = $1) < 0 -- negative balance

```

3.7.1 One remittance, worked through

STEP	PAISE	RUPEES	WHAT IT MEANS
<code>expected_paise</code> , 41 delivered COD shipments	374500	3,745.00	Summed from <code>shipments.cod_paise</code> . Written when the weekly cron opens the row as DUE.
<code>declared_paise</code> , entered by the fleet	374500	3,745.00	Veer Logistics agrees with our figure. Row moves to DECLARED.
<code>received_paise</code> , seen on the statement	366000	3,660.00	The credit is short by one delivery's worth of cash. Row moves to DEPOSITED.
<code>variance_paise</code>	-8500	-85.00	366000 - 374500. Negative, so the platform is out of pocket.
<code>variance_bps</code>	227	—	ROUND(8500 * 10000 / 374500). Above the 200 bps threshold, so this is not a rounding difference anyone waves through.
resulting status	SHORT	—	Terminal. Offers are blocked, the <code>fleet_ledger</code> carries <code>COD_HELD_DEBIT -374500</code> against <code>COD_SETTLED_CREDIT +366000</code> and therefore sits at -8500, and resolution is a new remittance row plus a compensating entry, never an edit of this one.

Two rules block a fleet, and only one of them is about variance. A single week inside the threshold still blocks work if the running `fleet_ledger` sum is negative, because a fleet that is 1.5 % short four weeks running has taken real money and no individual week trips the ratio. The ledger sum is the memory the per-row variance does not have.

3.8 HOW PERSONAL DATA IS STORED

Three columns per sensitive field: cipher, last four, and an HMAC index.

A masked field is not one column. It is up to three: **x_cipher** holds AES-256-GCM ciphertext with the nonce and tag prefixed, **x_last4** holds the last four characters in plaintext because that is what every screen in every portal displays, and **x_index** holds an HMAC-SHA256 of the normalised value so an exact-match lookup is possible without decrypting anything. The original column is not kept. AA-08 is the authority on who may see what; this section is only about what is on disk.

Storing last4 separately rather than deriving it on read is deliberate. Deriving it means decrypting on every list render, which puts the key in the hot path of the busiest admin screen and makes a masked list slower than an unmasked one — an incentive pointing the wrong way. With last4 as its own column, the Prisma masking extension can serve a whole page of rows without touching the key at all, and the key is used only on write and on the one manifest read.

FIELD	WHERE IT LIVES	STORAGE	EXACT-MATCH LOOKUP
Customer mobile	users.phone	cipher + last4 + HMAC index	Yes. HMAC in x_index, unique.
Customer name	user_profiles.full_name	plaintext (not a masked field)	Not needed. Stored in the clear.
Delivery address	addresses.*	line1/line2 cipher; locality, city, pincode plaintext	No. Shown, never searched.
Delivery coordinates	addresses.geo_lat/lng	cipher, 3-decimal rounded on read	No. Shown, never searched.
Delivery OTP	delivery_jobs.otp_hash	bcrypt hash only, digits never stored	No. Verified by compare, never searched.
Login OTP	otp_challenges.code_hash	bcrypt hash only	No. Verified by compare, never searched.
Payment reference	payments.gateway_ref	cipher + last4	No. Shown, never searched.
Partner GSTIN / PAN	partner_orgs.*	cipher + last4 + HMAC index	Yes. HMAC in x_index, unique.
Bank account / IFSC	partner_bank_accounts, fleet_bank_accounts	cipher + last4	No. Shown, never searched.
Payout UTR	payout_runs.utr_reference	cipher + last4	No. Shown, never searched.
Rider licence number	riders.dl_*	cipher + last4 + HMAC index	Yes. HMAC in x_index, unique.
Vehicle registration	vehicles.reg_*	cipher + last4 + HMAC index	Yes. HMAC in x_index, unique.
KYC document image	Cloudinary private folder	signed URL, 5-minute expiry	Not applicable. No database column.
Staff / partner / fleet email	staff_users, partner_users, fleet_users	plaintext	Not needed. Stored in the clear.

Note the two rows that are deliberately not encrypted. A customer's name is plaintext because a name is not a credential and masking it makes the admin order list unusable for support; what is restricted is who sees the full name versus the first name, and that is an authorisation rule, not a storage one. Business email addresses are plaintext for the same reason. Pretending otherwise would have produced a schema that looks stricter and a portal that is not.

WHY AN HMAC INDEX AND NOT SEARCHABLE ENCRYPTION

The only query anyone needs against a masked field is exact match: find the customer with this phone number, find the rider with this licence number. Deterministic encryption would work but reuses the key for the searchable value and leaks equality across the whole table with the same key that protects the plaintext. Order-preserving encryption leaks ordering for a range query nobody asked for. A proper searchable-encryption scheme needs a client-side index and a second key-management story. An HMAC-SHA256 with PII_INDEX_HMAC_KEY, a key separate from PII_ENC_KEY, gives exactly equality and nothing else, costs one hash on write and one on lookup, and is a unique index Postgres can plan.

KEY MANAGEMENT, WHICH IS THE WEAK POINT AND IS NOT SOLVED

PII_ENC_KEY is a 32-byte key in a Cloud Run secret environment variable. There is no KMS, no envelope encryption and no automatic rotation, because managed KMS is not in the Rs 0 budget. Rotation is a manual re-encrypt job: read every cipher column, decrypt with the old key, encrypt with the new, write back, in batches, with the application in read-only. PII_INDEX_HMAC_KEY is worse — rotating it invalidates every x_index column, so it is documented as never rotated. Both are named gaps in AA-08 and neither is scheduled inside these ten weeks. The honest position is that the encryption protects against a leaked database dump and not against a compromised api instance.

3.9 PII_DISCLOSURES: THE REGISTER BEHIND THE ONE EXCEPTION

Same transaction as the read, or no read. Nine columns, five reasons.

The masking regime has exactly one hole in it by design, and this table is what makes the hole accountable. GET /fleet/jobs/:id/manifest releases the delivery address and coordinates to a fleet principal for a job that fleet has accepted, inside a window of 240 minutes, and writes a pii_disclosures row in the same transaction. The customer's phone number is not part of that release and appears in no reason code in the table below: a rider who needs to call a customer does so through a masked bridge, not by reading a number.

The columns are id, actor_kind, actor_id, subject_user_id, field, reason_code, ref_type, ref_id and created_at. subject_user_id is the person whose data was read, which is the column that makes a subject-access request answerable in one query. ref_type and ref_id say what justified it — for a manifest, ("delivery_job", the job id) — so a disclosure can always be walked back to a specific piece of work.

```
// packages/shared/pii.ts – the only function in the codebase that returns a decrypted value.
// There is no exported decrypt(). If you need plaintext, you write a register row.
export async function discloseOrThrow(tx, { actorKind, actorId, subjectUserId,
                                         field, reasonCode, ref }) {
  assertReasonAllowedFor(actorKind, reasonCode); // 403 DISCLOSURE_REASON_FORBIDDEN
  await assertWindowOpen(tx, reasonCode, ref);   // 403 DISCLOSURE_WINDOW_CLOSED
  await tx.piiDisclosure.create({ data: { actorKind, actorId, subjectUserId,
                                         field, reasonCode, refType: ref.type,
                                         refId: ref.id } });
  return decryptField(tx, subjectUserId, field); // same tx: the row and the read commit together
}

// The contract in one sentence: if the transaction rolls back, the disclosure did not happen
// AND the caller got nothing. There is no ordering in which one exists without the other.
```

REASON CODE	WHEN IT IS WRITTEN	ACTOR	WHAT IS RELEASED
DELIVERY_MANIFEST	A fleet principal opened the manifest for an accepted job.	fleet	address, coordinates, receiver name
KYC_REVIEW	A staff reviewer opened a KYC document image.	staff	document image
SUPPORT_ESCALATION	A step-up-authorized ops action needed the address to resolve a dispute.	staff	address
LEGAL_REQUEST	A documented law-enforcement or regulatory request was served.	staff	any field, named in the row
EXPORT_JOB	A masked CSV export was generated. Recorded even though it is masked.	staff	nothing raw; recorded for completeness

EXPORT_JOB is in the list even though an export contains only masked values. It is recorded because the question an auditor asks is not "who saw plaintext" but "who took a copy of the data out of the system", and a masked CSV of five thousand rows is still a copy. Including it costs one row per export and removes a whole category of argument.

Retention is twenty-four months and nothing deletes these rows. That is a floor rather than a ceiling, stated as such because the two statements in the catalogue — "retained 24 months" and "never purged by a job" — read as a contradiction and are not one. The audit_log archival cron that AA-09 relies on to keep Neon under 0.5 GB explicitly excludes pii_disclosures: it is the smallest of the audit tables and the one whose absence would be hardest to explain.

3.10 INDEXES, AND THE QUERIES THAT JUSTIFY THEM

Five screens drive most of the index list. Partial indexes do most of the work.

Every index below exists because a named screen or cron would otherwise scan a table. Where the query has a status filter the index is partial, which matters more than usual here: a partial index on open work stays small enough to be cached even as the underlying table grows, and small indexes are the cheapest thing we can do about a 0.5 GB storage cap.

```
-- 1. Admin exception board (AA-05): jobs nobody has taken, oldest first.
CREATE INDEX dj_unassigned_age ON delivery_jobs (created_at)
  WHERE status IN ('DRAFT', 'OFFERED', 'UNASSIGNED');

-- 2. Fleet offer inbox (AA-07): live offers for one fleet, soonest to expire first.
CREATE INDEX jo_inbox ON job_offers (fleet_org_id, expires_at) WHERE status = 'OFFERED';
--   Service-area filtering is a join to fleet_service_areas, indexed on (fleet_org_id, pincode).

-- 3. Partner shipment queue (AA-06): one supplier, open work, ordered by its own promise.
CREATE INDEX sh_partner_queue ON shipments (partner_org_id, status, ready_by);

-- 4. Settlement run scan: accrual lines not yet in a run. Two tables, one shape.
CREATE INDEX pp_accrued ON partner_payouts (partner_org_id, id) WHERE status = 'ACCRUED';
CREATE INDEX fp_accrued ON fleet_payouts (fleet_org_id, id) WHERE status = 'ACCRUED';

-- 5. Advisor SLA breach list: open requests past due, which is the queue header count.
CREATE INDEX ar_sla_open ON advisory_requests (sla_due_at)
  WHERE status IN ('QUEUED', 'CLAIMED');

-- Supporting: the ledger fast path from 3.6.1, exact-match PII lookup from 3.8, subject access.
CREATE INDEX pl_latest ON partner_ledger (partner_org_id, id DESC);
CREATE INDEX fl_latest ON fleet_ledger (fleet_org_id, id DESC);
CREATE UNIQUE INDEX users_phone_index ON users (phone_index);
CREATE INDEX pd_subject ON pii_disclosures (subject_user_id, created_at DESC);
-- Plus one index per tenant column under R4, and the UNIQUE on delivery_jobs.shipment_id
-- that enforces one job per shipment. Those are mechanical and are not listed individually.
```

3.10.1 What the free tier actually does to these plans

Neon Free gives 0.5 GB of storage and roughly 190 compute-hours a month, and it suspends the compute after five minutes with no connections. Two consequences follow, and only one of them is a problem.

The storage cap is the one to watch, and it is a schema problem rather than an index problem: `order_events`, `audit_log` and the three ledgers are the tables that grow without bound, and AA-09 sets the alert at 350 MB with `audit_log` archival as the release valve. Indexes are a second-order contributor, which is why the partial ones above are written the way they are.

The autosuspend is mostly arithmetic. A permanently-awake endpoint at the minimum 0.25 compute unit costs about 180 compute-hours in a 720-hour month, which fits inside 190 with very little to spare — so the budget tolerates the database being awake, but it does not tolerate a larger compute size. In practice the operational crons in AA-09, the most frequent of which sweeps expiring offers every two minutes, keep the compute awake through the working day as a side effect of doing real work.

A synthetic keep-alive ping is deliberately not used. It would consume the same compute-hours as the real crons while doing nothing, and it would remove the only lever available if the monthly figure creeps up, which is to thin out the out-of-hours cron schedule. Instead the crons that only matter during the 08:00 to 20:00 operating window are scheduled inside it, the database is allowed to suspend overnight, and the first staff login of the morning pays the resume. Neon documents a sub-second resume; the planning figure is 500 ms to 1 s of extra latency on that first request, it has not been measured on this project yet, and measuring it is a week 9 task. The portals render skeletons rather than a blank frame precisely so a cold start reads as a slow screen and not a broken one, and the api retries once on a connection error so a resume never surfaces as a 500.

3.11 THE EIGHT MIGRATIONS, IN THE ORDER THEY MUST BE APPLIED

M1 to M8. One of them is dangerous and it is not the biggest one.

The order is not a preference. M3 cannot make `products.partner_org_id` NOT NULL until M2 has backfilled it; M6 cannot create `delivery_jobs` until M4 has created the shipments they point at and M5 has created the fleet orgs they may be offered to; M7 cannot accrue a payout line until both tenant roots exist. M8 is last for the opposite reason: it depends on nothing, so putting it last keeps an audit-table change off the critical path of a money migration.

M	WHAT IT CREATES	TABLES AND ALTERATIONS	BACKFILL	NOTES
M1	backoffice identity	<code>staff_users</code> , <code>staff_roles</code> , <code>staff_role_grants</code> , <code>role_permissions</code> , <code>staff_totp</code>	None. Purely additive.	Seeds the 9 system roles and the 88 permission rows. Creates the first <code>SUPER_ADMIN</code> from <code>BOOTSTRAP_ADMIN_EMAIL</code> , which the migration then requires to enrol TOTP on first login.
M2	partner tenancy	<code>partner_orgs</code> , <code>partner_users</code> , <code>partner_kyc</code> , <code>partner_bank_accounts</code> , <code>partner_rate_cards</code>	Adds nullable <code>products.partner_org_id</code> .	Backfills every existing product to the seeded house org <code>ATV-HOUSE</code> so the column can be made NOT NULL in M3 without downtime.
M3	catalog ownership	<code>ALTER products.partner_org_id SET NOT NULL</code> ; new partial index on <code>products (partner_org_id, moderation_status)</code>	Requires M2 backfill complete.	Adds <code>products.moderation_status</code> <code>DRAFT</code> <code>IN_REVIEW</code> <code>APPROVED</code> <code>REJECTED</code> <code>DELISTED</code> with default <code>APPROVED</code> for existing rows only, then flips the column default to <code>DRAFT</code> .
M4	shipment split	<code>shipments</code> , <code>shipment_items</code> , <code>order_events</code>	THE risky one. Backfills one shipment per existing order.	Runs in a single transaction over a table that is small at MVP. The backfill assigns every existing order a single shipment against <code>ATV-HOUSE</code> , copies status via a mapping table, and writes synthetic <code>order_events</code> from the existing status timestamps.
M5	fleet tenancy	<code>fleet_orgs</code> , <code>fleet_users</code> , <code>fleet_kyc</code> , <code>fleet_bank_accounts</code> , <code>fleet_service_areas</code> , <code>riders</code> , <code>vehicles</code>	None. Additive.	Seeds one internal fleet org <code>ATV-SELF</code> at tier 1 so orders can still be delivered by staff while no third-party fleet has onboarded.
M6	delivery jobs	<code>delivery_jobs</code> , <code>job_offers</code> , <code>pod_records</code>	Requires M4 (needs <code>shipment_id</code>) and M5 (needs <code>fleet_org_id</code>).	Creates one <code>delivery_job</code> per existing shipment in a terminal state, marked <code>legacy_backfill = true</code> , so historical orders render in the fleet timeline without offers.
M7	settlement	<code>partner_payouts</code> , <code>payout_runs</code> , <code>partner_ledger</code> , <code>fleet_payouts</code> , <code>fleet_ledger</code> , <code>cod_remittances</code>	Requires M2 and M5.	Installs the three append-only Postgres rules. Does NOT backfill historical payouts — the first run covers the first full week after go-live only, stated explicitly in the release note.
M8	governance	<code>pii_disclosures</code> , <code>audit_log</code> , <code>idempotency_keys</code> , <code>saved_views</code> , <code>exports</code> , <code>notification_templates</code> , <code>webhook_deliveries</code>	None. Additive.	Last because it is the only migration that can be applied at any time; it is deliberately kept out of the critical path so an audit-log schema change never blocks a money migration.

M	MUST RUN AFTER	ONLINE, OR A WRITE PAUSE	THE QUERY THAT PROVES IT LANDED
M1	Nothing.	Online. Pure <code>CREATE TABLE</code> .	<code>SELECT count(*) FROM staff_roles</code> ; expect 9. <code>SELECT count(*) FROM role_permissions</code> ; expect the full grant set. One <code>staff_users</code> row, zero <code>staff_totp</code> rows.
M2	M1, for the staff user who reviews KYC.	Online. The new column is nullable and the backfill is a single <code>UPDATE</code> over a small table.	<code>SELECT count(*) FROM products WHERE partner_org_id IS NULL</code> ; must be 0 before M3 runs.
M3	M2 backfill complete.	Brief lock. <code>SET NOT NULL</code> takes <code>ACCESS EXCLUSIVE</code> and normally scans; adding a <code>NOT VALID CHECK</code> , validating it, then setting <code>NOT NULL</code> lets Postgres skip the scan and hold the lock for milliseconds.	<code>SELECT attnotnull FROM pg_attribute WHERE attrelid = 'products'::regclass AND attname = 'partner_org_id'</code> ; expect true. Plus the <code>moderation_status</code> default is <code>DRAFT</code> , not <code>APPROVED</code> .
M4	M2, for <code>ATV-HOUSE</code> as the shipment owner.	<code>WRITE PAUSE</code> . A 15-minute read-only window, three rehearsals on a restored Neon branch first, runbook RB-11 for the rollback. This is risk R-01.	The two zero-row assertions below, plus a diff of every stored <code>orders.status</code> against <code>reduceOrderStatus()</code> over the backfilled shipments. Zero rows or the window is used to roll back.
M5	M1.	Online. Additive, plus one seeded row.	<code>SELECT tier FROM fleet_orgs WHERE code = 'ATV-SELF'</code> ; expect 1. No other fleet org exists at this point.

M	MUST RUN AFTER	ONLINE, OR A WRITE PAUSE	THE QUERY THAT PROVES IT LANDED
M6	M4 for shipment_id, M5 for fleet_org_id.	Online. The backfill inserts one terminal job per existing shipment and touches no live row.	One job per shipment, asserted below. SELECT count(*) FROM delivery_jobs WHERE legacy_backfill AND status NOT IN ('DELIVERED','RTO_DELIVERED','CANCELLED'); expect 0.
M7	M2 and M5.	Online, with a brief lock per ledger table while the trigger is attached. No historical backfill at all.	Three triggers present in pg_trigger. Then the negative test: an UPDATE against each ledger must raise LEDGER_IMMUTABLE, asserted in the migration itself and in CI.
M8	Nothing. Deliberately last.	Online. Pure CREATE TABLE.	Table existence only. This is the one migration whose failure blocks nothing else.

3.11.1 M4: the backfill and the assertions that gate the commit

M4 first adds orders.legacy_status and copies orders.status into it, which is what makes the mapping below auditable and the rollback in RB-11 possible; the column is kept for two releases and then merely stops being read. Everything that follows runs in one transaction with the api serving reads only.

```
-- 1. One shipment per existing order, owned by the seeded house org, status mapped.
INSERT INTO shipments (id, code, order_id, partner_org_id, status, item_count,
                      goods_paise, cod_paise, packed_at, handed_over_at, delivered_at, created_at)
SELECT gen_random_uuid(), 'SHP-' || o.code, o.id, h.id, map.shipment_status,
       (SELECT count(*) FROM order_items oi WHERE oi.order_id = o.id),
       o.goods_paise, o.cod_paise, o.packed_at, o.handed_over_at, o.delivered_at, o.created_at
FROM orders o
CROSS JOIN (SELECT id FROM partner_orgs WHERE code = 'ATV-HOUSE') h
JOIN legacy_status_map map ON map.order_status = o.legacy_status;

-- 2. Every order line assigned in full to that one shipment. No splitting on the way in.
INSERT INTO shipment_items (shipment_id, order_item_id, qty, created_at)
SELECT s.id, oi.id, oi.qty, now()
FROM shipments s JOIN order_items oi ON oi.order_id = s.order_id;

-- 3. One synthetic timeline row per shipment, from data we already had. Nothing invented.
INSERT INTO order_events (order_id, shipment_id, event_code, actor_kind, payload_json, created_at)
SELECT s.order_id, s.id, 'MIGRATED_FROM_LEGACY', 'system',
       jsonb_build_object('legacy_status', o.legacy_status), o.created_at
FROM shipments s JOIN orders o ON o.id = s.order_id;
```

```
-- Gate the COMMIT on all four returning zero rows. Any row and the transaction rolls back.
SELECT o.id FROM orders o LEFT JOIN shipments s ON s.order_id = o.id
WHERE s.id IS NULL;           -- every order has a shipment
SELECT order_id FROM shipments GROUP BY order_id HAVING count(*) <> 1;  -- exactly one
SELECT oi.id FROM order_items oi LEFT JOIN shipment_items si ON si.order_item_id = oi.id
GROUP BY oi.id, oi.qty HAVING COALESCE(SUM(si.qty), 0) <> oi.qty;        -- lines fully assigned
SELECT o.id FROM orders o JOIN shipments s ON s.order_id = o.id
WHERE reduce_order_status(ARRAY[s.status], o.payment_state) <> o.legacy_status;

-- The row-count assertion, stated as an equality and checked again after M6:
-- count(orders) = count(shipments) = count(delivery_jobs WHERE legacy_backfill)
-- It holds only for pre-migration orders. Every order created after M4 may break it, and
-- that is the point: from then on an order may legitimately have more than one shipment.
```

3.12 SEED AND FIXTURE DATA

What a developer needs locally before any of the three portals is usable.

An empty schema is not a working environment: the admin portal needs a staff user per role to test a permission boundary, and the supplier and fleet portals need two tenants each or a cross-tenant leak is invisible. One idempotent seed script, run after all eight migrations, produces the set below.

FIXTURE	COUNT	WHAT IT IS AND WHY THAT NUMBER
partner_orgs	3	ATV-HOUSE from M2, plus Sagar Agro Traders and Krishi Bhandar. Two real tenants is the minimum that makes a cross-tenant read fail visibly rather than return an empty list.
fleet_orgs	3	ATV-SELF at tier 1 from M5, plus Veer Logistics and Nayak Carriers at tiers 2 and 3, so the three offer waves can each be observed.
staff_users	9	One per role, each with a staff_totp row seeded from a fixed base32 secret that is printed once by the script, so a developer can enrol all nine in an authenticator in one pass.
partner_users and fleet_users, riders, vehicles, service areas	2 each; 3, 2, 6 per fleet	An owner and a staff-level user per org, to exercise the within-tenant role split. Six pincodes overlapping between the two external fleets so an offer genuinely has two candidates.
catalogue slice	12 products, 4 categories	Includes one licence-gated pesticide and one item heavy enough to price a pickup, because both are branches in the shipment and job paths.
orders	4	One single-supplier, one two-supplier, one COD, one already delivered. These four are what make reduceOrderStatus() observable in a browser rather than only in a unit test.

Two rules bind the seed script. Phone numbers are synthetic, drawn from a documented non-routable block, and the script asserts no seeded value collides with an existing phone_index before it writes. And this script is the single place in the codebase where a *_cipher column is written by something other than the api: it imports encrypt() from packages/shared/pii.ts rather than reimplementing AES-GCM, so there is one implementation and one key path. It writes no pii_disclosures rows, because it never reads anything back.

3.13 WHAT THIS SCHEMA DOES BADLY

Six real limitations. None of them is scheduled to be fixed in these ten weeks.

LIMITATION	WHY IT IS LIKE THIS	WHEN IT BITES, AND WHAT WOULD FIX IT
order_events is unpartitioned and absorbs job events too	One timeline table was chosen over separate order and job event tables so the customer tracker is a single indexed read. That makes it the fastest-growing table in the database.	Comfortable to a few million rows; past that, the tracker read and the audit scan both degrade and it dominates the 0.5 GB cap. The fix is declarative partitioning by month, which is a table rewrite and therefore forbidden by R1 inside this programme. Phase two, with a stated write pause.
Ledger re-derivation is O(rows)	balance_after_paise makes a balance read one row, but the proof that it is right is a window function over every row for that org.	The nightly assertion in 3.6.1 gets slower every month and there is no checkpoint to start from. Monthly opening-balance rows would bound it; they are not built.
No soft delete on catalogue rows	moderation_status DELISTED is the only tombstone, and products are never removed because historical order lines join to them.	Every read of products must exclude DELISTED, and one forgotten filter puts a delisted SKU back in a listing. A repository default helps; it does not make the mistake impossible.
The HMAC index cannot do prefix or partial search	x_index is a hash, so it answers equality and nothing else — the deliberate trade in 3.8 against searchable encryption.	Support cannot look up "the customer whose number starts 98" and never will. The admin search empty state has to say so in words, or an ops team concludes search is broken.
Additive-only accumulates dead columns by design	R1 forbids dropping anything for ten weeks, so every superseded column stays.	orders.packed_at, orders.handed_over_at and orders.legacy_status are dead the day M4 lands. They cost little storage and a lot of confusion, and the mitigation is a comment in the Prisma schema plus a phase-two cleanup migration nobody has scheduled.
shipment_items allows a split it cannot express twice	The composite PK is (shipment_id, order_item_id), so one order line can be spread across shipments but not across two shipments of the same supplier.	A re-split that keeps both halves with one supplier has to synthesise a second order_items row, so the order line count differs from what the customer saw at checkout. It is rare, it is ugly, and it is documented rather than fixed.

WHAT TO DO WITH THIS LIST

Each item is a decision with a named reason and a named cost, not a bug waiting to be found. The first person to hit one should recognise it and know that the fix is a phase-two migration with a write pause. A seventh limitation discovered during the build belongs here, in this document, and not in a commit message nobody will read again.